WORKSHOP · SESSION 1

OAuth 2.0 — the real flow, in Go

Learn OAuth by running a real go-oidc demo end to end.

RUN THE LAB

```
cd resource/labs/day1-go-oauth-demo
go run .
open http://localhost:8082
```

- Auth Code + PKCE
- Refresh Token
- Client Credentials
- Device Authorization

SESSION 1 OUTCOME

Explain the flow without hiding behind JWT

1. Map the four OAuth actors and the endpoints onto the running app.
2. Draw Authorization Code + PKCE from memory.
3. Explain browser / front-channel vs back-channel requests.
4. Explain state, redirect_uri, code_challenge, and code_verifier.
5. Choose the right flow for common application shapes.

THE DEMO APP

One Go program, three separate servers

Real network separation — each OAuth role is its own HTTP server (see main.go).

Authorization Server

`:8080 · op.go`

go-oidc OpenID Provider

`/authorize /token`
`/introspect`
`/device_authorization`

Resource Server API

`:8081 · api.go`

Protected endpoints

`/api/profile`
`/api/fraud-report`
`/api/deploy`

Client App

`:8082 · client.go`

RepoBoard + service + CLI

`/start /callback`
`/refresh /call-api`
`START HERE → :8082`

TEACHING RHYTHM

How we teach each flow

Concept	What problem are we solving?
Diagram	Who talks to whom, and on which channel?
Go code	Which function in client.go / op.go / api.go?
Browser flow	What URL or token actually appears?
Checkpoint	Can participants explain it back?

Participants run and read the demo — they do not write code in Session 1.

THE PROBLEM

Limited access without the password

RepoBoard needs to read the user's GitHub-like profile — without ever getting their password.

Password sharing — wrong shape

- User hands RepoBoard a password
- App can do everything the user can
- No way to scope to one resource
- Revoking means changing the password

OAuth delegation — right shape

- User approves limited API access
- Client receives a scoped token, not a password
- Access can be revoked on its own
- Token expires automatically

The client receives permission, not the user's password.

Map the four actors onto the code

Resource Owner	Mock user “Alya Developer” (Subject = alya)
Client	:8082 · client.go — RepoBoard (client_id repoboard-web)
Authorization Server	:8080 · op.go — go-oidc provider (login, consent, tokens)
Resource Server	:8081 · api.go — protected APIs, validates tokens

Ask the room: which actor issues the token, and which actor validates it?

ENDPOINT MAP

/authorize and /token are not interchangeable

/authorize (:8080, front-channel)

- Browser redirect — the user sees it
- Login + consent happen here
- Returns: code + state in the redirect

/token (:8080, back-channel)

- Server-to-server POST — no redirect
- Exchanges code (+ verifier) for tokens
- Returns: JSON token response

Both endpoints are served by go-oidc `(op.Handler())` — you do not implement them. Your client calls them from client.go.

AUTHORIZATION CODE + PKCE

The browser gets a code; the client gets tokens

1	**Client**	`/start  (:8082)`
2	**Authorize**	`/authorize  (:8080)`
3	**Callback**	`/callback?code&state  (:8082)`
4	**Token**	`POST /token  (:8080)`
5	**API**	`GET /api/profile  (:8081)`

Demo anchor

Click “Start Auth Code + PKCE flow” on :8082, then ask participants what travels through the browser vs the back-channel.

START AUTHORIZATION

Client builds the request before redirecting

client.go

```
func (c *ClientApp) StartAuthCode(w, r) {
  state         := randomString(18)
  codeVerifier  := randomString(48)
  codeChallenge := pkceChallenge(codeVerifier)
  // save state + verifier in the session

  q := url.Values{}
  q.Set("response_type", "code")
  q.Set("client_id", "repoboard-web")
  q.Set("redirect_uri", clientBaseURL+"/callback")
  q.Set("scope", "openid profile profile.read offline_access")
  q.Set("state", state)
  q.Set("code_challenge", codeChallenge)
  q.Set("code_challenge_method", "S256")
  http.Redirect(w, r, opBaseURL+"/authorize?"+q.Encode(), 302)
}
```

What to point at

- state + code_verifier are created on the client, before any redirect.
- Only the code_challenge (a hash) goes through the browser.
- The verifier stays in the session — it is never sent to /authorize.

AUTHORIZATION REQUEST

Every parameter in the redirect, explained

`response_type=code`	Ask for an authorization code
`client_id`	Identify the client app (repoboard-web)
`redirect_uri`	Where the browser returns (:8082/callback)
`scope`	Permissions requested
`state`	Callback binding — CSRF protection
`code_challenge (+ _method=S256)`	PKCE challenge for the code exchange

USER APPROVAL

Login & consent is a go-oidc policy

op.go

```
loginPolicy := goidc.NewPolicy("simple_login",
  func(...) bool { return true },   // applies to all
  func(w, r, as, _) (goidc.Status, error) {
    if r.Method == GET {
      // render the consent screen
      return goidc.StatusPending, nil
    }
    if r.FormValue("approve") == "true" {
      as.Subject       = "alya"
      as.GrantedScopes = as.Scopes
      return goidc.StatusSuccess, nil
    }
    return goidc.StatusFailure, nil
  })
```

What to point at

- This is the hook where a real app would check username / password / MFA.
- go-oidc creates and stores the authorization code internally on success.
- The code is short-lived and is not an access token.

CALLBACK

Verify state, then exchange the code

client.go

```
func (c *ClientApp) Callback(w, r) {
  code  := r.URL.Query().Get("code")
  state := r.URL.Query().Get("state")
  if state == "" || state != expectedState {
    http.Error(w, "state mismatch — possible CSRF", 400)
    return
  }
  // back-channel POST to /token
  postForm(opBaseURL+"/token", url.Values{
    "grant_type":    {"authorization_code"},
    "client_id":     {"repoboard-web"},
    "code":          {code},
    "redirect_uri":  {clientBaseURL + "/callback"},
    "code_verifier": {codeVerifier},
  })
}
```

What to point at

- The callback arrives through the browser (front-channel).
- state is checked first — reject anything the client did not start.
- The code-for-token exchange is back-channel, and carries the code_verifier.

TOKEN EXCHANGE

go-oidc verifies the exchange for you

At /token, go-oidc checks:

- The code exists and has not been used
- client_id and redirect_uri match the original request
- SHA-256(code_verifier) equals the stored code_challenge
- Then it issues access + refresh tokens

op.go

```
// enabled by provider options in op.go
provider.WithAuthCodeGrant(store, ResponseTypeCode),
provider.WithPKCE(CodeChallengeMethodSHA256),
provider.WithRefreshTokenGrant(store),
provider.WithRefreshTokenRotation(),
```

Your client never implements PKCE verification — it is a configuration choice on the server.

It binds the exchange to the original request

`At /authorize`	go-oidc stores client_id, redirect_uri, scope, state, code_challenge.
`At /callback`	The browser returns code + state to the client.
`At /token`	The client resends redirect_uri + code_verifier; go-oidc checks they match.

The token endpoint does not redirect — it verifies the exchange belongs to the same request.

PKCE MENTAL MODEL

Challenge first, verifier later

Before redirect (client)

- Create code_verifier (random secret)
- Send code_challenge = hash of verifier

At /token (client → server)

- Client sends the code_verifier
- Server hashes it and compares

```
code_challenge = BASE64URL( SHA256(code_verifier) )      // pkceChallenge() in client.go
```

SHA-256 is one-way: the server repeats the hash and compares — it never reverses it.

The resource server validates by introspection

api.go

```
func apiProfile(w, r) {
  info, ok := introspect(r, "profile.read")
  if !ok {
    writeJSON(w, 401, ...)   // invalid token or scope
    return
  }
  // return profile JSON
}

// introspect(): POST /introspect (Basic auth),
// require active == true AND the needed scope
```

What to point at

- The API (:8081) does not share memory with the AS (:8080).
- It validates the bearer token by calling /introspect — the real decoupled pattern.
- A token for the wrong API or missing scope is rejected.

REFRESH TOKEN FLOW

Renew access without re-authorizing

How the demo does it

- client.go Refresh() posts grant_type=refresh_token
- Rotation is on: a new refresh token replaces the old one
- Refresh tokens are issued only when offline_access scope is granted

Why it matters

- Access tokens should be short-lived
- Refresh tokens are more sensitive — they mint new access tokens
- Rotation makes refresh-token reuse detectable

CLIENT CREDENTIALS FLOW

Service-to-service — no user, no browser

How the demo does it

- order-service authenticates with Basic auth (id : secret)
- grant_type=client_credentials, scope fraud.check
- Then calls /api/fraud-report with the token

Key idea

- No browser redirect, no consent screen
- The token represents the service itself, not a user
- A confidential client can safely hold a secret

DEVICE AUTHORIZATION FLOW

Approve on a second device while the CLI polls

How the demo does it

- DeviceStart() → POST /device_authorization (deploy-cli, deploy.write)
- User opens the verification page and approves
- DevicePoll() polls /token with the device_code

What to point at

- Good for CLI, TV, and limited-input devices
- Before approval: authorization_pending
- After approval: token response, then calls /api/deploy

CHOOSE THE FLOW

Start from the application shape

Web / SPA / mobile with a user	**Authorization Code + PKCE**
Backend service, no user	**Client Credentials**
Renew an access token	**Refresh Token**
CLI / TV / limited input	**Device Authorization**
Need the user's identity / login	**OpenID Connect**

Two questions, two tokens

OAuth 2.0

- Delegated API access
- Access token
- Scopes; the resource API validates

OpenID Connect

- Login identity
- ID token
- User claims; the client validates identity

Heads-up for Session 2: this demo requests the `openid` scope, so go-oidc also issues an `id_token` — that is OpenID Connect, covered in Session 2.

COMMON SESSION 1 MISTAKES

Most confusion mixes tokens, actors, and endpoints

Calling OAuth “login”	Use OIDC language when identity / login is the goal.
Access token as identity	Access token is for APIs; ID token carries login claims.
Thinking /token redirects	/token returns JSON data to the client; it never redirects.
Confusing state and PKCE	state protects the callback; PKCE protects the code exchange.

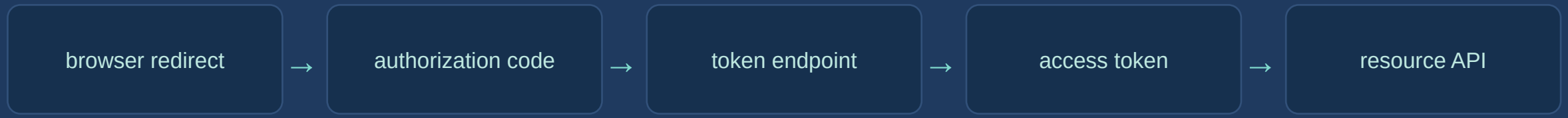
INSTRUCTOR DEMO CHECKLIST

Run it in this exact order

1. `go run .  (starts :8080, :8081, :8082)`
2. `Open http://localhost:8082`
3. “Start Auth Code + PKCE flow” → Approve for Alya Developer
4. “Call resource API with current access token”
5. “Use refresh token”
6. “Run Client Credentials demo”
7. “Start Device Authorization demo” → open verification page → approve → poll
8. “Reset All Data” to run a clean pass again

SESSION 1 CHECKPOINT

Explain this chain from memory



Final task: say who sends each request, where the token appears, and which party validates it.